

Multicast Delegates in C# with Examples

In this article, I am going to discuss the **Multicast Delegates in C#** with Examples. Please read our previous article where we discussed **Delegates in C#** with Examples. As part of this article, we are going to discuss the following pointers in detail.

1. **What is Multicast Delegate in C#?**
2. **How to Create Multicast Delegates in C#?**
3. **Different ways to Create Multicast Delegates.**
4. **Multicast Delegate with Return Type in C#.**
5. **Multicast Delegate with Output Parameter in C#.**

What is a Delegate in C#?

A Delegate is a Type-Safe Function Pointer. It means the delegate holds the reference of a method or function and when we invoke the delegate, the method it refers to is going to be executed. The delegate signature and the method it points to must have the same signature. While creating the delegate instance, we need to pass the method as a parameter to the delegate constructor. Again, there are two types of Delegates in C#. They are as follows:

1. **Single Cast Delegate:** Delegate Refers to a single function or method.
2. **Multicast Delegate:** Delegate Refers to multiple functions or methods.

We have already discussed **Singlecast Delegate** in our previous article. Today, we are going to discuss Multicast Delegates in C#.

What is Multicast Delegate in C#?

A Multicast Delegate in C# is a delegate that holds the references of more than one handler function. When we invoke the multicast delegate, then all the functions which are referenced by the delegate are going to be invoked. If you want to call multiple methods using a delegate then all the method signatures should be the same.

So, a multicast delegate is just an array of multiple pipelines or multiple delegates. The delegates are going to be invoked in the same order as they are placed in the invocation list. An InvocationList is nothing but an array of delegates or pipelines where each pipeline will dump data into a different method. If this is not clear at the moment don't worry, we will try to understand this with multiple examples.

Example to Understand Multicast Delegate in C#

Let us see an example to understand the Multicast Delegate in C#. Please have a look at the following example which is without using delegates. In the below example, we created two methods, and then from the main method, we are creating the instance of the class and then invoke the two methods.

```
using System;
namespace MulticastDelegateDemo
{
    public class Rectangle
```

```

{
    public void GetArea(double Width, double Height)
    {
        Console.WriteLine($"Area is {Width * Height}");
    }
    public void GetPerimeter(double Width, double Height)
    {
        Console.WriteLine($"Perimeter is {2 * (Width + Height)}");
    }
    static void Main(string[] args)
    {
        Rectangle rect = new Rectangle();
        rect.GetArea(23.45, 67.89);
        rect.GetPerimeter(23.45, 67.89);
        Console.ReadKey();
    }
}

```

Output:

```

Area is 1592.0205
Perimeter is 182.68

```

In the above example, we created an instance of the Rectangle class and then called the two methods. Now I want to create a single delegate that is going to invoke the above two methods (i.e. GetArea and GetPerimeter). The two methods have the same signature with different method names, so we can create a single delegate that holds the reference of the above two methods. And when we invoke the delegate, it is going to invoke the above two methods. And when we do so, then it is called a Multicast Delegate in C#.

Multicast Delegate Example in C#:

In the below example, we have created one delegate whose signature is the same as the two methods i.e. GetArea and GetPerimeter. Then we created the instance of delegate and bind the two methods using the += operator. Similarly, you can use the -= operator to remove a function from the delegate. Once we bind the two methods with the delegate instance and when we call the delegate, both methods are going to be executed. In this case, behind the scene, when we add multiple methods to the delegate, then multiple pipelines are added. In other words, we can now say that the InvocationList now contains two delegates or two pipelines in the same order we add the methods. In this case, the first delegate or pipeline will dump the data into the GetArea method and the second pipeline will dump the data into the GetPerimeter method, and when you will run the application, then you will see that the GetArea method is first executed, and then the GetPerimeter method is going to be executed. And in InvocationList, you will see that we have two pipelines or delegates having the same name in this example.

```

using System;
namespace MulticastDelegateDemo
{
    public delegate void RectangleDelegate(double Width, double Height);
    public class Rectangle
    {

```

```

public void GetArea(double Width, double Height)
{
    Console.WriteLine($"Area is {Width * Height}");
}
public void GetPerimeter(double Width, double Height)
{
    Console.WriteLine($"Perimeter is {2 * (Width + Height)}");
}
static void Main(string[] args)
{
    Rectangle rect = new Rectangle();
    RectangleDelegate rectDelegate = new RectangleDelegate(rect.GetArea);
    // RectangleDelegate rectDelegate = rect.GetArea;
    // binding a method with delegate object
    // In this example rectDelegate is a multicast delegate.
    // You use += operator to chain delegates together.
    rectDelegate += rect.GetPerimeter;
    Delegate[] InvocationList = rectDelegate.GetInvocationList();
    Console.WriteLine("InvocationList:");
    foreach (var item in InvocationList)
    {
        Console.WriteLine($" {item}");
    }
    Console.WriteLine();
    Console.WriteLine("Invoking Multicast Delegate:");
    rectDelegate(23.45, 67.89);
    //rectDelegate.Invoke(13.45, 76.89);

    Console.WriteLine();
    Console.WriteLine("Invoking Multicast Delegate After Removing one
Pipeline:");
    //Removing a method from delegate object
    rectDelegate -= rect.GetPerimeter;
    rectDelegate.Invoke(13.45, 76.89);
    Console.ReadKey();
}
}
}

```

Output:

```
InvocationList:
  MulticastDelegateDemo.RectangleDelegate
  MulticastDelegateDemo.RectangleDelegate
```

Invoking Multicast Delegate:

Area is 1592.0205

Perimeter is 182.68

Invoking Multicast Delegate After Removing one Pipeline:

Area is 1034.1705

Another Approach to Creating Multicast Delegates in C#:

In the below example, I am going to show you the use of both static and non-static methods along with different ways to create and invoke multicast delegates in C#. Please have a look at the below example. Here, we created one delegate which takes two integer parameters and returns nothing. Then within the program class, we define four methods and all these four methods take two integer parameters and return nothing i.e. void. Then we created four instances of the delegate and bind the four methods. Finally, we create the fifth delegate instance and to this instance, we are binding all four delegate instances using the + operator. Now, the fifth delegate becomes a multicast delegate. In this case, for delegate 5, the InvocationList has 4 delegates or you can say 4 pipelines where each pipeline or delegate is going to dump the data into a different method. And when we invoke the fifth delegate instance then all four methods are going to be executed. If you want to remove one method binding, then simply you need to use the -= operator and specify the delegate instance which you want to remove.

```
using System;
namespace MulticastDelegateDemo
{
    public delegate void MathDelegate(int No1, int No2);
    public class Program
    {
        //Static Method
        public static void Add(int x, int y)
        {
            Console.WriteLine($"Addition of {x} and {y} is : {x + y}");
        }
        //Static Method
        public static void Sub(int x, int y)
        {
            Console.WriteLine($"Subtraction of {x} and {y} is : {x - y}");
        }
        //Non-Static Method
        public void Mul(int x, int y)
        {
            Console.WriteLine($"Multiplication of {x} and {y} is : {x * y}");
        }
        //Non-Static Method
        public void Div(int x, int y)
        {
            Console.WriteLine($"Division of {x} and {y} is : {x / y}");
        }
    }
}
```

```

    }
    static void Main(string[] args)
    {
        Program p = new Program();
        //Static Method within the same class can be access directly
        MathDelegate del1 = new MathDelegate(Add);
        //Static Method can be access using class name
        MathDelegate del2 = new MathDelegate(Program.Sub);
        //Non-Static Method must be access through object
        MathDelegate del3 = new MathDelegate(p.Mul);
        MathDelegate del4 = new MathDelegate(p.Div); ;
        //In this example del5 is a multicast delegate.
        //We can use +(plus) operator to chain delegates together and
        //- (minus) operator to remove a delegate.
        MathDelegate del5 = del1 + del2 + del3 + del4;
        Delegate[] InvocationList = del5.GetInvocationList();
        Console.WriteLine("InvocationList:");
        foreach (var item in InvocationList)
        {
            Console.WriteLine($" {item}");
        }
        Console.WriteLine();
        Console.WriteLine("Invoking Multicast Delegate::");
        del5.Invoke(20, 5);
        Console.WriteLine();
        Console.WriteLine("Invoking Multicast Delegate After Removing one
Delegate:");
        del5 -= del2;
        del5(22, 7);
        Console.ReadKey();
    }
}

```

Output:

```
InvocationList:
MulticastDelegateDemo.MathDelegate
MulticastDelegateDemo.MathDelegate
MulticastDelegateDemo.MathDelegate
MulticastDelegateDemo.MathDelegate

Invoking Multicast Delegate::
Addition of 20 and 5 is : 25
Subtraction of 20 and 5 is : 15
Multiplication of 20 and 5 is : 100
Division of 20 and 5 is : 4

Invoking Multicast Delegate After Removing one Delegate:
Addition of 22 and 7 is : 29
Multiplication of 22 and 7 is : 154
Division of 22 and 7 is : 3
```

Multicast Delegates with Return Type in C#:

A multicast delegate invokes the methods in the same order in which they are added to the Invocation List and that we have already seen. As of now, the examples we have discussed using Multicast delegate, are not returning anything i.e. the return type is void. Now, what happens if the delegate has a return type?

If the delegate has a return type other than void, and if the delegate is a multicast delegate, then only the value of the last invoked method will be returned. Along the same lines, if the delegate has an out parameter, the value of the output parameter will be the value assigned by the last invoked method from the Invocation List.

Example to Understand Multicast Delegates with Return Type in C#

Let's understand Multicast Delegates with Return Type in C# with an example. Please have a look at the below example. Here, we created one delegate which does not take any parameter but its return type is int. Then we created two static methods, the first static method returns 1 and the second static method returns 2. Then we created the delegate instance and first we bind MethodOne and then we bind MethodTwo. And when we invoke the delegate, first MethodOne is executed and then MethodTwo is executed and it returns 2 as the last invoked method in the InvocationList is MethodTwo which returns 2.

```
using System;
namespace MulticastDelegateDemo
{
    // Delegate's return type is int
    public delegate int SampleDelegate();
    public class Program
    {
        static void Main()
        {
            SampleDelegate del = new SampleDelegate(MethodOne);
            del += MethodTwo;
        }
    }
}
```

```

        // The Value Returned By Delegate will be 2, returned by the Methc
        // as it is the last method in the invocation list.
        int ValueReturnedByDelegate = del();
        Console.WriteLine($"Returned Value = {ValueReturnedByDelegate}");
        Console.ReadKey();
    }
    // This method returns one
    public static int MethodOne()
    {
        Console.WriteLine("MethodOne is Executed");
        return 1;
    }
    // This method returns two
    public static int MethodTwo()
    {
        Console.WriteLine("MethodTwo is Executed");
        return 2;
    }
}

```

Output:

```

MethodOne is Executed
MethodTwo is Executed
Returned Value = 2

```

Example to Understand Multicast Delegates using out Parameter in C#:

Now we will see an example of a Multicast delegate in C# using the out parameter. Please have a look at the below example. Here, we created one delegate which takes one out parameter and returns nothing i.e. void. Then we created two static methods and both the static methods take one out parameter. The first static method assigns value 1 to the out parameter and the second static method assigns value 2 to the out parameter. Then we created the delegate instance and first bind MethodOne and then bind MethodTwo. And when we invoke the delegate, first MethodOne is executed and then MethodTwo is executed and it returns 2 as the last invoked method is MethodTwo which assigns value 2 to the out parameter.

```

using System;
namespace MulticastDelegateDemo
{
    // Delegate has an int output parameter
    public delegate void SampleDelegate(out int Integer);
    public class Program
    {
        static void Main()
        {
            SampleDelegate del = new SampleDelegate(MethodOne);
            del += MethodTwo;
            // The ValueFromOutputParameter will be 2, initialized by MethodTv
            // as it is the last method in the invocation list.

```

```

        int ValueFromOutPutParameter = -1;
        del(out ValueFromOutPutParameter);
        Console.WriteLine($"Returned Value = {ValueFromOutPutParameter}");
        Console.ReadKey();
    }
    // This method sets ouput parameter Number to 1
    public static void MethodOne(out int Number)
    {
        Console.WriteLine("MethodOne is Executed");
        Number = 1;
    }
    // This method sets ouput parameter Number to 2
    public static void MethodTwo(out int Number)
    {
        Console.WriteLine("MethodTwo is Executed");
        Number = 2;
    }
}

```

Output:

```

MethodOne is Executed
MethodTwo is Executed
Returned Value = 2

```

In the next article, I am going to discuss one [Real-time Example of using Delegates in C#](#). Here, In this article, I try to explain **Multicast Delegates in C#** with Examples. I hope you enjoy this article and understand the need and use of Multicast Delegates in C# with Examples.